

Assignment-02 Report

[FSStat]

Programmazione Concorrente e Distribuita [25-26]

Alex Mazzoni

`alex.mazzoni3@studio.unibo.it`

May 19, 2026

Contents

1	Analisi del Problema	2
1.1	Asincronia e ricorsione	2
1.2	Composizione dei risultati (Report)	2
1.3	Gestione Eccezioni e directory non leggibili	2
1.4	Aggiornamenti dinamici e backpressure	2
2	Design e Implementazione	3
2.1	Struttura del report	3
2.2	Event-loop: Vert.x	3
2.3	Programmazione reattiva: RxJava	3
2.3.1	Implementazione Dinamica con GUI	4
2.4	Virtual threads	4

1 Analisi del Problema

L'obiettivo dell'assignment è progettare una libreria **FSStatLib** capace di produrre asincronamente un report sulle dimensioni dei file contenuti in una directory, includendo ricorsivamente tutte le sottodirectory. Il problema è apparentemente semplice: visitare un albero, leggere la dimensione dei file e aggiornare dei contatori. Dal punto di vista concorrente però diventa interessante perché la visita del file system è un'attività fortemente I/O-bound, con profondità e ampiezza non note a priori.

1.1 Asincronia e ricorsione

La chiamata alla libreria non deve congelare il chiamante mentre l'albero viene esplorato. Questo è particolarmente importante come nel caso della versione **Rx**, in cui avremo che il client è una GUI. 2.3

La difficoltà non è solo 'spostare il lavoro su un thread'. Effettuare una scansione ricorsiva, permette ogni directory di essere esplorata indipendentemente, e per ogni file produrre un piccolo contributo al report. Il design deve quindi permettere parallelismo, evitando l'utilizzo di una struttura globale condivisa, tra tutti i thread.

1.2 Composizione dei risultati (Report)

Ogni directory visitata produce un **Report** e i risultati vengono poi fusi tramite **merge** tra le altre directory. Ogni task lavora su una propria istanza locale e la composizione avviene solo quando il padre raccoglie i risultati dei figli.

Questa scelta riduce il rischio di race e mantiene la logica del **Report** riutilizzabile, la concorrenza sta nell'esplorazione dell'albero, non nella gestione dello stato condiviso. In questo modo si adotta un design più semplice, senza appunto dover introdurre meccanismi di sincronizzazione complessi.

1.3 Gestione Eccezioni e directory non leggibili

Durante lo sviluppo della versione **event-loop**, è emerso il primo problema. In caso in cui si presentino: una directory non leggibile o un path speciale, è necessario gestirli e non far fallire l'intero report.

Le implementazioni gestiscono i nodi non leggibili e restituiscono un report vuoto per quel sottoalbero.

1.4 Aggiornamenti dinamici e backpressure

Il punto opzionale richiede aggiornamenti dinamici e la possibilità di stop. È stato implementato nella versione **Rx**, poichè l'utilizzo del pattern **Observable** ha reso piuttosto semplice l'intercettazione dei singoli report nel tempo.

Il problema principale durante lo sviluppo del punto opzionale, è stato il disaccoppiamento tra produttore e consumatore. Come accennato a lezione, la scansione può produrre aggiornamenti molto più velocemente di quanto Swing possa ridisegnare la

GUI (consumare). Senza una politica esplicita si accumulano eventi inutili oppure si degrada la reattività. Per questo la pipeline usa **Flowable**, `onBackpressureLatest()` e un campionamento temporale prima del rendering.

2 Design e Implementazione

Le tre versioni condividono solo il concetto di **Report**, ma sono state progettate con design differenti. Ho evitato di creare un'astrazione unica generica, perché avrebbe nascosto proprio le unicità di design tra **event-loop**, **programmazione reattiva** e **virtual threads**.

2.1 Struttura del report

Il **Report** contiene un numero specifico di fasce **NB**, il suo scopo è quello di tenere traccia del numero di file caduti in ciascuna fascia. L'ultima fascia rappresenta i file con dimensione maggiore o uguale a **MaxFS**. Il metodo `addFile` aggiorna il bucket, mentre `merge` somma due report con lo stesso numero di fasce (NB).

I dati del report sono rappresentati con due vettori paralleli, i ranges visualizzati come stringa (es. `[0, 250)`, ovvero 0 incluso e 250 escluso) e per ogni range il numero di file che rientrano in quella fascia. I dati sono poi serializzati in JSON per essere visualizzati sia in console che in GUI.

2.2 Event-loop: Vert.x

`Future<Report>`. L'attraversamento della directory viene espresso come composizione di operazioni asincrone: prima si leggono le proprietà del path, poi, se il path è una directory, si leggono i figli e si compongono le loro **Promise**, traducendone in una singola Promise di **Report**.

Il punto più importante di questa versione è non bloccare mai l'event-loop. Il codice non chiama `join`, `get` o sleep bloccanti. La ricorsione viene modellata con l'aggregazione delle promise (`compose` e `all`). Quando tutti i figli di una directory hanno completato, il padre crea un **Report** locale e fonde i report dei figli.

Gli errori di accesso a un singolo nodo 1.3 sono gestiti con `recover`, producendo un report vuoto, senza interrompere l'intera elaborazione.

2.3 Programmazione reattiva: RxJava

La scansione viene rappresentata come un flusso di file e successivamente mappata come un flusso di report progressivi. La funzione che genera l'albero restituisce un `Flowable<File>`, ogni file incontrato viene accumulato tramite `scan`, così il client può osservare non solo il report finale ma anche gli stati intermedi.

2.3.1 Implementazione Dinamica con GUI

Il design di Rx contiene anche il punto opzionale. La GUI Swing si iscrive alla pipeline e riceve aggiornamenti dinamici del grafico. Per evitare che il rendering venga saturato, la pipeline usa `onBackpressureLatest()` e la GUI applica `sample(20, TimeUnit.MILLISECONDS, true)`: se arrivano molti report intermedi, viene mantenuto solo quello più recente.

Lo stop è implementato tramite il `Disposable` restituito da `subscribe`. In questo modo l'interruzione non viene gestita con flag condivisi sparsi nel codice, ma sfruttando il modello di cancellazione della pipeline reattiva.

2.4 Virtual threads

La versione con `vthreads` usa `Executors.newVirtualThreadPerTaskExecutor()` e restituisce un `CompletableFuture<Report>`. La chiamata pubblica è asincrona: il client ottiene subito un future, mentre la visita del file system avviene in virtual thread.

Per ogni figlio di una directory viene creato un task sullo stesso executor; Il task padre aspetta i figli con `join` e poi fonde tutti i report parziali. In una soluzione con platform thread questa forma rischierebbe di produrre thread bloccati in attesa. Invece quando un virtual thread attende, il carrier thread può essere riutilizzato dalla JVM.